

CHAPTER 7

Text Processing

7.1 Regular Expression Syntax

Some text processing programs, such as *grep*, *egrep*, *sed*, *awk* and *vi*, let you search on patterns instead of fixed strings. These text patterns are known as **regular expressions**. You form a regular expression by combining normal characters and special characters, also known as **meta-characters**, with the rules below. With these regular expressions you can do **pattern matching** on text data. Regular expressions come in three different forms:

- **Anchors** which tie the pattern to a location on the line
- **Character sets** which match a character at a single position
- **Modifiers** which specify how many times to repeat the previous expression

Regular expression syntax is as follows. Some programs will accept all of these, others may only accept some.

.	match any single character except <newline>
*	match zero or more instances of the single character (or meta-character) immediately preceding it
[abc]	match any of the characters enclosed
[a-d]	match any character in the enclosed range
[^exp]	match any character not in the following expression
^abc	the regular expression must start at the beginning of the line (Anchor)
abc\$	the regular expression must end at the end of the line (Anchor)
\	treat the next character literally. This is normally used to escape the meaning of special characters such as "." and "*".
\{n,m\}	match the regular expression preceding this a minimum number of n times and a maximum of m times (0 through 255 are allowed for n and m). The \{ and \} sets should be thought of as single operators. In this case the \ preceding the bracket does not escape its special meaning, but rather turns on a new one.
\<abc\>	will match the enclosed regular expression as long as it is a separate word. Word boundaries are defined as beginning with a <newline> or anything except a letter, digit or underscore (_) or ending with the same or a end-of-line character. Again the \< and \> sets should be thought of as single operators.

\(abc\)	saves the enclosed pattern in a buffer. Up to nine patterns can be saved for each line. You can reference these latter with the \n character set. Again the \(and \) sets should be thought of as single operators.
\n	where n is between 1 and 9. This matches the nth expression previously saved for this line. Expressions are numbered starting from the left. The \n should be thought of as a single operator.
&	print the previous search pattern (used in the replacement string)

There are a few meta-characters used only by *awk* and *egrep*. These are:

+	match one or more of the preceding expression
?	match zero or more of the preceding expression
 	separator. Match either the preceding or following expression.
()	group the regular expressions within and apply the match to the set.

Some examples of the more commonly used **regular expressions** are:

regular expression	matches
cat	the string cat
.at	any occurrence of a letter, followed by at , such as cat , rat , mat , bat , fat , hat
xy*z	any occurrence of an x , followed by zero or more y 's, followed by a z .
^cat	cat at the beginning of the line
cat\$	cat at the end of the line
*	any occurrence of an asterisk
[cC]at	cat or Cat
^[a-zA-Z]	any occurrence of a non-alphabetic character
[0-9]\$	any line ending with a number
[A-Z][A-Z]*	one or more upper case letters
[A-Z]*	zero or more upper case letters (In other words, anything.)

7.2 Text Processing Commands

TABLE 7.1 Text Processing Commands

Command/Syntax	What it will do
<i>awk/nawk</i> [options] <i>file</i>	scan for patterns in a file and process the results
<i>grep/egrep/fgrep</i> [options] 'search string' <i>file</i>	search the argument (in this case probably a file) for all occurrences of the search string, and list them.
<i>sed</i> [options] <i>file</i>	stream editor for editing files from a script or from the command line

7.2.1 grep

This section provides an introduction to the use of **regular expressions** and *grep*.

The *grep* utility is used to search for generalized regular expressions occurring in Unix files. Regular expressions, such as those shown above, are best specified in apostrophes (or single quotes) when specified in the *grep* utility. The *egrep* utility provides searching capability using an extended set of meta-characters. The syntax of the *grep* utility, some of the available options, and a few examples are shown below.

Syntax

grep [options] regexp [file[s]]

Common Options

- i** ignore case
- c** report only a count of the number of lines containing matches, not the matches themselves
- v** invert the search, displaying only lines that do not match
- n** display the line number along with the line on which a match was found
- s** work silently, reporting only the final status:
 - 0, for match(es) found
 - 1, for no matches
 - 2, for errors
- l** list filenames, but not lines, in which matches were found

Examples

Consider the following file:

```
{unix prompt 5} cat num.list
 1      15   fifteen
 2      14   fourteen
 3      13   thirteen
 4      12   twelve
 5      11   eleven
 6      10   ten
 7       9    nine
 8       8    eight
 9       7    seven
10       6     six
11       5     five
12       4     four
13       3     three
14       2     two
15       1     one
```

Here are some *grep* examples using this file. In the first we'll search for the number **15**:

```
{unix prompt 6} grep '15' num.list
 1      15   fifteen
15       1     one
```

Now we'll use the **"-c"** option to count the number of lines matching the search criterion:

```
{unix prompt 7} grep -c '15' num.list
2
```

Here we'll be a little more general in our search, selecting for all lines containing the character **1** followed by either of **1**, **2** or **5**:

```
{unix prompt 8} grep '1[125]' num.list
 1      15   fifteen
 4      12   twelve
 5      11   eleven
11       5     five
12       4     four
15       1     one
```

Now we'll search for all lines that **begin** with a **space**:

```
{unix prompt 9} grep '^ ' num.list
```

1	15	fifteen
2	14	fourteen
3	13	thirteen
4	12	twelve
5	11	eleven
6	10	ten
7	9	nine
8	8	eight
9	7	seven

Or all lines that **don't begin** with a **space**:

```
{unix prompt 10} grep '^[^ ]' num.list
```

10	6	six
11	5	five
12	4	four
13	3	three
14	2	two
15	1	one

The latter could also be done by using the **-v** option with the original search string, e.g.:

```
{unix prompt 11} grep -v '^ ' num.list
```

10	6	six
11	5	five
12	4	four
13	3	three
14	2	two
15	1	one

Here we search for all lines that **begin** with the characters **1 through 9**:

```
{unix prompt 12} grep '^[1-9]' num.list
```

10	6	six
11	5	five
12	4	four
13	3	three
14	2	two
15	1	one

This example will search for any instances of **t** followed by **zero or more** occurrences of **e**:

```
{unix prompt 13} grep 'te*' num.list
```

```
1      15   fifteen
2      14   fourteen
3      13   thirteen
4      12   twelve
6      10    ten
8       8    eight
13     3     three
14     2     two
```

This example will search for any instances of **t** followed by **one or more** occurrences of **e**:

```
{unix prompt 14} grep 'tee*' num.list
```

```
1      15   fifteen
2      14   fourteen
3      13   thirteen
6      10    ten
```

We can also take our input from a program, rather than a file. Here we report on any lines output by the *who* program that begin with the letter **l**.

```
{unix prompt 15} who | grep '^l'
```

```
lcondron tty0 Dec 1 02:41 (lcondron-pc.acs.)
```

7.2.2 sed

The non-interactive, stream editor, *sed*, edits the input stream, line by line, making the specified changes, and sends the result to standard output.

Syntax

sed [options] edit_command [file]

The format for the editing commands are:

[address1[,address2]][function][arguments]

where the addresses are optional and can be separated from the function by spaces or tabs. The function is required. The arguments may be optional or required, depending on the function in use.

Line-number Addresses are decimal line numbers, starting from the first input line and incremented by one for each. If multiple input files are given the counter continues cumulatively through the files. The last input line can be specified with the "\$" character.

Context Addresses are the regular expression patterns enclosed in slashes (/).

Commands can have 0, 1, or 2 comma-separated addresses with the following affects:

# of addresses	lines affected
0	every line of input
1	only lines matching the address
2	first line matching the first address and all lines until, and including, the line matching the second address. The process is then repeated on subsequent lines.

Substitution functions allow context searches and are specified in the form:

s/regular_expression_pattern/replacement_string/flag

and should be quoted with single quotes (') if additional options or functions are specified. These patterns are identical to context addresses, except that while they are normally enclosed in slashes (/), any normal character is allowed to function as the delimiter, other than <space> and <newline>. The replacement string is not a regular expression pattern; characters do not have special meanings here, except:

&	substitute the string specified by regular_expression_pattern
\n	substitute the nth string matched by regular_expression_pattern enclosed in '(' , ')' pairs.

These special characters can be escaped with a backslash (\) to remove their special meaning.

Common Options

-e script	edit script
-n	don't print the default output, but only those lines specified by p or s///p functions
-f script_file	take the edit scripts from the file, script_file

Valid flags on the substitution functions include:

d	delete the pattern
g	globally substitute the pattern
p	print the line

Examples

This example changes all incidents of a comma (,) into a comma followed by a space (,) when doing output:

```
% cat file | sed s/,,\ /g
```

The following example removes all incidents of **Jr** preceded by a space (**Jr**) in **file**:

```
% cat file | sed s/\ Jr//g
```

To perform multiple operations on the input precede each operation with the **-e** (edit) option and quote the strings. For example, to filter for lines containing "Date: " and "From: " and replace these without the colon (:), try:

```
sed -e 's/Date: /Date /' -e 's/From: /From /'
```

To print only those lines of the file from the one beginning with "Date:" up to, and including, the one beginning with "Name:" try:

```
sed -n '/^Date:/,/^Name:/p'
```

To print only the first 10 lines of the input (a replacement for *head*):

```
sed -n 1,10p
```


7.2.3 **awk, nawk, gawk**

awk is a pattern scanning and processing language. Its name comes from the last initials of the three authors: Alfred. V. Aho, Brian. W. Kernighan, and Peter. J. Weinberger. **nawk** is **new awk**, a newer version of the program, and **gawk** is **gnu awk**, from the Free Software Foundation. Each version is a little different. Here we'll confine ourselves to simple examples which should be the same for all versions. On some OSs **awk** is really **nawk**.

awk searches its input for patterns and performs the specified operation on each line, or fields of the line, that contain those patterns. You can specify the pattern matching statements for **awk** either on the command line, or by putting them in a file and using the **-f program_file** option.

Syntax

awk program [file]

where **program** is composed of one or more:

pattern { action }

fields. Each input line is checked for a pattern match with the indicated action being taken on a match. This continues through the full sequence of patterns, then the next line of input is checked.

Input is divided into **records** and **fields**. The default **record** separator is **<newline>**, and the variable **NR** keeps the record count. The default **field** separator is whitespace, **spaces** and **tabs**, and the variable **NF** keeps the field count. Input field, **FS**, and record, **RS**, separators can be set at any time to match any single character. Output field, **OFS**, and record, **ORS**, separators can also be changed to any single character, as desired. **\$n**, where **n** is an integer, is used to represent the **n**th field of the input record, while **\$0** represents the entire input record.

BEGIN and **END** are special patterns matching the beginning of input, before the first field is read, and the end of input, after the last field is read, respectively.

Printing is allowed through the **print**, and formatted print, **printf**, statements.

Patterns may be regular expressions, arithmetic relational expressions, string-valued expressions, and boolean combinations of any of these. For the latter the patterns can be combined with the boolean operators below, using parentheses to define the combination:

 	or
&&	and
!	not

Comma separated patterns define the **range** for which the pattern is applicable, e.g.:

/first/,/last/

selects all lines starting with the one containing **first**, and continuing inclusively, through the one containing **last**.

To select lines 15 through 20 use the pattern range:

NR == 15, NR == 20

Regular expressions must be enclosed with slashes (/) and meta-characters can be escaped with the backslash (\). Regular expressions can be grouped with the operators:

	or, to separate alternatives
+	one or more
?	zero or one

A regular expression match can be either of:

~	contains the expression
!~	does not contain the expression

So the program:

\$1 ~ /[Ff]rank/

is true if the first field, \$1, contains "Frank" or "frank" anywhere within the field. To match a field identical to "Frank" or "frank" use:

\$1 ~ /^[Ff]rank\$/

Relational expressions are allowed using the relational operators:

<	less than
<=	less than or equal to
==	equal to
>=	greater than or equal to
!=	not equal to
>	greater than

Offhand you don't know if variables are strings or numbers. If neither operand is known to be numeric, then string comparisons are performed. Otherwise, a numeric comparison is done. In the absence of any information to the contrary, a string comparison is done, so that:

\$1 > \$2

will compare the string values. To ensure a numerical comparison do something similar to:

(\$1 + 0) > \$2

The **mathematical functions**: exp, log and sqrt are built-in.

Some other **built-in functions** include:

index(s,t)	returns the position of string s where t first occurs, or 0 if it doesn't
length(s)	returns the length of string s
substr(s,m,n)	returns the n -character substring of s , beginning at position m

Arrays are declared automatically when they are used, e.g.:

```
arr[i] = $1
```

assigns the first field of the current input record to the *i*th element of the array.

Flow control statements using **if-else**, **while**, and **for** are allowed with **C** type syntax:

```
for (i=1; i <= NF; i++) {actions}
while (i<=NF) {actions}
if (i<NF) {actions}
```

Common Options

-f program_file	read the commands from program_file
-Fc	use character c as the field separator character

Examples

```
% cat filex | tr a-z A-Z | awk -F: '{printf ("7R  %-6s %-9s  %-24s \n", $1, $2, $3)}'>upload.file
```

cats filex, which is formatted as follows:

```
nfb791:99999999:smith
7ax791:99999999:jones
8ab792:99999999:chen
8aa791:99999999:mcnulty
```

changes all lower case characters to upper case with the **tr** utility, and formats the file into the following which is written into the file **upload.file**:

```
7R NFB791 99999999 SMITH
7R 7AX791 99999999 JONES
7R 8AB792 99999999 CHEN
7R 8AA791 99999999 MCNULTY
```